

Week06 | 课时 2 | Dagster Assets 与资产图：从任务流到资产流

本课导航

Asset graph 不是流程图，而是数据产品的运行边界图	1
这节课解决什么问题	1
参考学习时间	1
学完这一讲，你应该能做到什么	2
本课产出	2
asset graph 到底在画什么？	2
先看一张最小资产图	2
Dagster 核心概念：人话解释 + 工程解释	3
1. Week06 asset key 怎么命名	3
2. 什么该 materialize，什么该 observe	4
什么该变成 asset，什么不该	4
为什么 source observation 不能和 core materialization 混选？	5
Asset card 怎么写	5
3. Definitions 是什么	6
4. Resources / config 不能硬编码	7
不要做什么	7
5. 本课最重要判断	7
自检清单	7
课后最小行动	8
延伸阅读	8
Footnotes	8

Asset graph 不是流程图，而是数据产品的运行边界图

这一讲把 Week06 的核心对象讲清楚：

asset key、依赖、Definitions、resources、external asset、observation 和 materialization 必须分清。

这节课解决什么问题

很多人第一次用 Dagster，会把每个 Python 函数都拆成 asset，或者把所有脚本直接串进一个 job。Week06 不能这么做。

这节课要解决：

- 哪些东西应该变成 asset；
- 哪些只是 source / external observation；
- asset key 如何避免和旧 assets 冲突；
- 为什么 source observation job 和 materialization job 不能混在一起。

参考学习时间

45—55 分钟

学完这一讲，你应该能做到什么

1. 读懂最小 Dagster asset graph。
2. 设计 `week06/*` 命名空间，避免 asset key 冲突。
3. 区分 observable source asset、external asset 和 materializable asset。
4. 解释 `Definitions` 为什么是 Dagster 入口。
5. 说明 `resources / config` 为什么不能硬编码本机路径。
6. 写出一张最小 Asset Card。

本课产出

- `docs/blueprints/week06/week06-asset-graph.md`
- `docs/blueprints/week06/week06-data-factory-blueprint.md`
- 一张 `week06/silver/ticket_fact_partitioned` 的 Asset Card

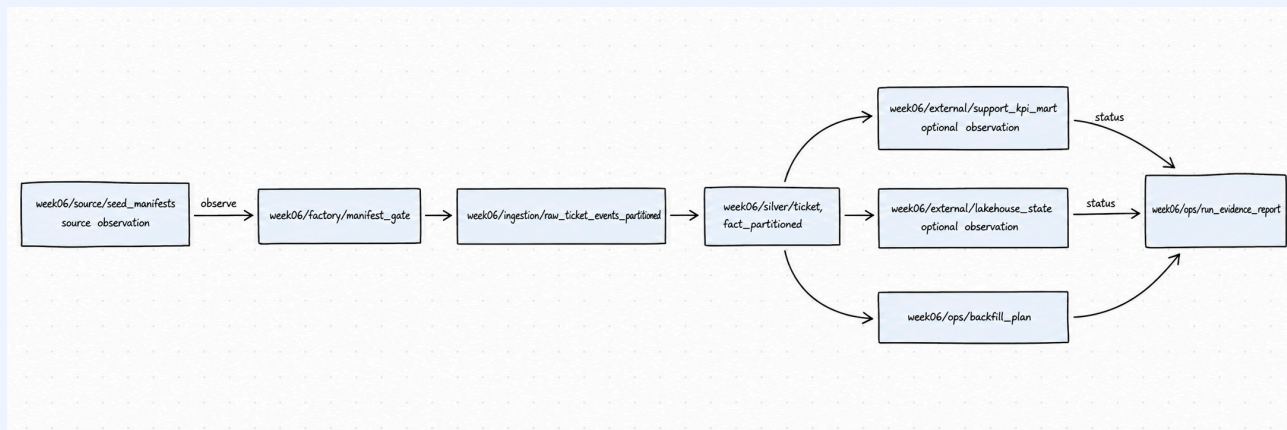
asset graph 到底在画什么？

普通流程图像导航路线：先去哪、再去哪、下一步怎么走。它适合描述“动作顺序”，但很难回答“现在货架上有什么、能不能卖、哪里缺货、谁批准放行”。

asset graph 更像库存和质检系统：每个货架上的商品是什么、从哪里来、当前能不能卖、失败后应该补哪个区域。对数据工程来说，它描述的不是函数调用顺序，而是数据产品之间的依赖、状态和可消费边界。

回到 Dagster：asset 是一个持久化结果或可观察的数据对象；asset definition 描述它如何被生成或观察；asset graph 则让团队看到这些对象之间的依赖、状态、检查和证据。¹

先看一张最小资产图



这张图有两个重点：

- source observation 可以观测输入，但不要把“看到了外部状态”误读成“本周资产已经成功产出”；
- Week04 / Week05 依赖可以先作为 optional / external，缺失时写 `not_available` 或 `skipped`，不要伪造通过。

¹ Dagster 的 asset 是软件定义资产，不只是任务节点。课程里把它解释为“团队要交付、维护、检查和交接的数据产品边界”。

Dagster 核心概念：人话解释 + 工程解释

概念	小白解释	工程解释	Week06 例子
Asset	一个团队要交付和维护的数据产品	可以被 <code>materialize</code> / <code>observe</code> / <code>check</code> 的对象	<code>week06/silver/ticket_fact_partitioned</code>
Asset key	资产稳定名字	<code>graph</code> 中定位资产的路径	<code>week06/ops/run_evidence_report</code>
Dependency	谁依赖谁	下游运行前需要哪些上游状态	<code>fact</code> 依赖 <code>raw</code>
Materialization	生成或更新资产	写入或产出当前状态，并可带 <code>metadata</code>	<code>week06/ops/run_evidence_report</code> 写证据 JSON ²
Observation	看外部状态	不修改资产，只记录 <code>metadata</code>	观察 Week05 <code>mart</code> 状态 ³
External asset	别的系统生产，本图要依赖	本周只观察或标注，不直接生产	<code>week06/external/lakehouse_state</code>
Definitions	Dagster 入口	注册 <code>assets</code> 、 <code>jobs</code> 、 <code>checks</code> 、 <code>resources</code>	<code>pipelines/definitions.py</code>
Resource	运行时连接	数据库、对象存储、配置、报告目录	<code>Postgres</code> / <code>MinIO</code> / <code>reports/week06/</code>

1. Week06 asset key 怎么命名

建议采用明确前缀：

资产	推荐 key	说明
seed manifests	<code>week06/source/seed_manifests</code>	读取本周可用 source manifest
manifest gate	<code>week06/factory/manifest_gate</code>	读取并校验本次输入声明
ticket raw asset	<code>week06/ingestion/raw_ticket_events_partitioned</code>	薄包装 Week03 ticket ingest
ticket fact asset	<code>week06/silver/ticket_fact_partitioned</code>	观察或物化结构化事实层
optional lakehouse	<code>week06/external/lakehouse_state</code>	未完成时 <code>skipped</code> / <code>not_available</code>
optional semantic mart	<code>week06/external/support_kpi_mart</code>	未完成时 <code>skipped</code> / <code>not_available</code>
backfill plan	<code>week06/ops/backfill_plan</code>	为一个 daily partition 生成 dry-run 计划

²Materialization 表示当前运行生成或更新了某个资产，并可以附带 `metadata`；它比普通日志更适合表达“这个资产现在是什么状态”。

³Observation 用来记录外部资产的状态，不代表本次运行修改了它。Week06 对 Week04 lakehouse 和 Week05 analytics 的 optional dependency 就应该优先用这种诚实语义。

资产	推荐 key	说明
run evidence	week06/ ops/run_evidence_report	汇总运行证据
delivery summary	week06/ops/ data_factory_delivery_summary	给实验 / 作业收口

命名原则：

- 使用 `week06/` 前缀隔离本周新增资产；
- 第二段表达 layer / domain，例如 `factory`、`ingestion`、`silver`、`external`、`ops`；
- 最后一段表达稳定资产名，不写临时动作；
- 不复用 Week03 / Week05 的旧 key，避免语义冲突；
- 不把 `run_this_script` 这种动作名当 asset key。

好命名	坏命名	原因
week06/factory/manifest_gate	run_manifest_script	asset key 应该描述数据产品边界，不是动作
week06/silver/ ticket_fact_partitioned	ticket_facts	需要 namespace 和分区语义，避免和旧表/旧资产混淆
week06/ external/support_kpi_mart	week05_success	external asset 要表达依赖对象，不表达主观状态
week06/ ops/run_evidence_report	final_report	evidence 是稳定交付物，不是课堂临时文件

不要直接复用旧的 `raw_ticket_events`、`ticket_facts` key，避免同名或语义接近的资产有两套依赖。

2. 什么该 materialize，什么该 observe

类型	适合对象	Week06 做法
materializable asset	本周负责生成或更新的资产	<code>manifest_gate</code> 、 <code>partitioned ticket assets</code> 、 <code>backfill plan</code> 、 <code>run evidence</code>
observable source asset	外部输入状态	<code>seed manifest</code> 、 <code>source files</code> ，可单独 job 或被 gate 读取
external asset	由其他系统负责，但本图需要依赖	<code>lakehouse</code> / <code>semantic mart</code>
placeholder asset	前置周末完成时保留接口	<code>status</code> 写 <code>skipped</code> 或 <code>not_available</code> ，不阻塞主线

什么该变成 asset，什么不该

对象	要不要做成 asset	理由
helper 函数	不要	它不是独立可消费数据产品，也不是恢复边界

对象	要不要做成 asset	理由
<code>manifest_gate</code>	要	决定输入是否能进入本次运行
<code>raw_ticket_events_partitioned</code>	要	有明确 daily partition 和恢复边界
<code>ticket_fact_partitioned</code>	要	下游会消费的结构化事实层
<code>support_kpi_mart</code>	optional / external	Week05 负责生产，本周只观察状态
<code>run_evidence_report</code>	要	它是 Week06 的验收证据

Student Core 不追求 asset 数量，追求少量关键资产边界清晰。一个 asset 应该回答：谁消费它，失败后补哪里，验收看什么证据。

为什么 source observation 不能和 core materialization 混选？

观察 source 是“看外部状态”，不代表本周资产被成功生成。materialization 是“本周负责产出”。如果把 source observation、external observation 和 core materialization 混在一个 job 里，学员很容易把“看到了”误解成“产出了”，也容易把 optional dependency 的缺失伪造成 passed。

⚠ 不要混选 source observation 和 core materialization

Week06 的 core job 只选择 regular materializable assets。source observation 可以单独定义 job，或在 Student Core 中让 `manifest_gate` 直接读取 manifest 文件。optional dependency 缺失时应该写 `not_available` 或 `skipped`，不能 fake passed。

💡 少量关键资产 + 薄包装

Student Core 不是把每个 helper 函数都变成 asset。优先把 manifest gate、partitioned raw、partitioned fact、backfill plan、run evidence、delivery summary 做清楚，再扩展 observation 和 external assets。

Asset card 怎么写

可复制模板：

```
## Asset Card: week06/silver/ticket_fact_partitioned

- asset_key:
- owner:
- group:
- partition_def:
- upstream:
- downstream:
- materialization_type:
- checks:
- metadata:
- evidence_path:
- failure_mode:
- recovery_action:
```

填好的示例：

```
## Asset Card : week06/silver/ticket_fact_partitioned

- asset_key: week06/silver/ticket_fact_partitioned
- owner: course-team / data-factory-owner
- group: week06_data_factory
- partition_def: daily partition, 默认课堂分区 2026-04-17
- upstream: week06/ingestion/raw_ticket_events_partitioned
- downstream: week06/ops/run_evidence_report, Week07 parse / Week08 retrieval baseline
- materialization_type: materializable asset
- checks: duplicate_idempotency, required_field_null_rate, partition_completeness,
row_count_output_count
- metadata: input_row_count, output_row_count, batch_id, dry_run, report_path
- evidence_path: reports/week06/run_evidence/*.json
- failure_mode: manifest missing, zero rows, duplicate ticket_id, required field nulls,
partition gap
- recovery_action: retry / replay / backfill dry-run / hold downstream
```

这张 Asset Card 不只是文档。它强迫你把资产名字、上下游、分区、检查、证据和恢复动作放在同一页，后续 runbook、assignment 和团队交接才不会散。

3. Definitions 是什么

Definitions 是 Dagster 的项目入口。Week06 不需要另起一套入口，而是在现有入口中注册：

```
from dagster import Definitions

defs = Definitions(
    assets=all_assets,
    asset_checks=all_asset_checks,
    jobs=[ingest_all_job, week06_data_factory_job],
    resources=build_week06_resources(),
)
```

课程重点不是背 API，而是理解：

- assets 定义“有哪些数据产品”；
- jobs 定义“哪些资产可以被一起运行”；
- resources 定义“运行时如何访问外部系统”；
- checks 定义“资产是否可验收”。⁴

项目当前真实落点：

对象	项目路径	用途
asset definitions	<code>pipelines/data_factory/assets.py</code>	定义 Week06 的 assets 和 materialization / observation 逻辑
asset keys	<code>pipelines/data_factory/asset_keys.py</code>	集中管理 week06/* asset key, 避免散落硬编码

⁴Asset checks 用来验证资产属性，例如行数、唯一性、必填字段、分区完整性。它不能替代单元测试，但能直接服务“这个资产能不能被下游消费”的判断。

对象	项目路径	用途
asset job	<code>pipelines/data_factory/jobs.py</code>	定义 <code>week06_data_factory</code> job
project entrypoint	<code>pipelines/definitions.py</code>	统一注册 assets、checks、jobs、resources

如果课程命令与项目 runbook 不一致，以项目 runbook 为准。

4. Resources / config 不能硬编码

Week06 需要的配置应来自 env 或 run config:

配置	示例
Database	<code>DATABASE_URL</code>
Object storage	<code>MINIO_ENDPOINT</code> 、bucket
Reports	<code>WEEK06_REPORT_DIR</code>
Partition start	<code>WEEK06_PARTITION_START_DATE</code>
Default partition	<code>WEEK06_DEFAULT_PARTITION</code>
Optional dependency	Week04 / Week05 evidence 是否存在

硬编码本机绝对路径会让 devbox、Dagster 容器和同学机器全部不一致。

可执行的项目侧落点以 `omnisupport-copilot` 当前 runbook 为准:

```
# 可执行：已与项目 runbook 对齐
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run
--rm devbox \
    pytest tests/integration/test_week06_definitions_loadable.py -q
```

看输出时只看一个判断：`week06_data_factory` job、`week06/*` assets、asset checks 和 resources 是否能一起被 Dagster definitions 加载。

不要做什么

- 不要新建一个平行的 Dagster entrypoint，Week06 继续走 `pipelines/definitions.py`。
- 不要复制 Week03 ingest 的 SQL / 写入逻辑，asset wrapper 只做薄包装。
- 不要因为 Week04 / Week05 optional 依赖未完成就写成 passed。
- 不要把每个 Python helper 拆成 asset，先保证少量资产能被解释、验证和交接。
- 不要把动作名写成 asset key，例如 `run_ticket_ingest`、`do_backfill_now`。

5. 本课最重要判断

Week06 的资产图要少而准：关键资产、清晰命名、明确依赖、可解释状态，比复杂图更重要。

自检清单

- ☐ 我能解释 asset graph 为什么不是普通流程图
- ☐ 我能解释 asset key 为什么要有 `week06/*` namespace

- ☐ 我能区分 source observation、external asset 和 materialization
- ☐ 我知道 Week04 / Week05 未完成时不能 fake passed
- ☐ 我能说出 [Definitions](#) 注册哪些对象
- ☐ 我知道哪些配置不能硬编码
- ☐ 我能为 `week06/silver/ticket_fact_partitioned` 写一张 Asset Card

课后最小行动

补一份 asset graph 草稿：

```
## Week06 asset graph draft

- Core materializable assets:
- Observable source assets:
- External / optional assets:
- Jobs:
- Required resources:
- Known skipped dependencies:
- One Asset Card:
```

延伸阅读

主题	推荐资料	为什么读
Dagster Assets	Dagster Docs: Assets	理解 asset graph 为什么表达数据资产，而不是普通流程图
Dagster Definitions	Dagster Docs: Definitions	理解 Dagster 项目入口如何注册 assets、jobs、checks、resources
Dagster Resources	Dagster Docs: Resources	理解为什么数据库、对象存储、报告目录不能硬编码
Dagster Asset Observations	Dagster Docs: Asset observations	理解 observe external asset 和 materialize core asset 的边界
Dagster Asset Checks	Dagster API: Asset checks	理解 checks 为什么是资产验收的一部分，而不是测试报告的替代品

Footnotes